Graph Traversal Algorithms

DFS, BFS, and the Mechanics of Network Exploration

1. Introduction to Graph Traversal

Traversing a graph involves visiting every vertex in a systematic manner. Unlike linear data structures, graphs require specialized algorithms to handle cycles and multi-directional paths. The two primary methods are **Depth First Search (DFS)** and **Breadth First Search (BFS)**.

> **Crucial Concept:** To avoid infinite loops in cyclic graphs, we must maintain a `visited` array (or set) to track which nodes have already been processed.

2. Depth First Search (DFS)

DFS explores as far as possible along each branch before backtracking. It behaves like exploring a maze by following a single path until a dead end is reached.

- **Data Structure:** Stack (usually the implicit System Call Stack via recursion).
- **Logic:** Visit node -> Mark visited -> Recursively visit all unvisited neighbors.

```
def dfs_util(self, v, visited):
    visited[v] = True
```

```
    print(self.vertex_data[v], end=' ')
    for i in range(self.size):
        if self.adj_matrix[v][i] == 1 and not visited[i]:
            self.dfs_util(i, visited)
```

3. Breadth First Search (BFS)

BFS explores the neighbor nodes first, before moving to the next level neighbors. It radiates outward from the starting point level-by-level.

- **Data Structure:** Queue (First-In, First-Out).
- **Logic:** Enqueue start -> While queue not empty: Dequeue -> Visit neighbors -> Enqueue unvisited neighbors.

```
def bfs(self, start_node):
    queue = [start_node]
    visited[start_node] = True
    while queue:
        current = queue.pop(0)
        # process current
        # add unvisited neighbors to queue
```

4. Comparison: DFS vs. BFS

Feature	DFS	BFS
Structure	Stack (Recursion)	Queue
Search Path	Deep (Distance increases)	Wide (Level-by-level)
Application	Pathfinding, Cycle detection	Shortest path (unweighted)
Memory	O(Height of Tree)	O(Width of Tree)

5. Traversing Directed Graphs

Both algorithms work on directed graphs with a single rule change: you can only follow edges in the direction they point. This may lead to some vertices being unreachable from certain starting points, even if the graph is "connected" in an undirected sense.

Deep Dive: The Call Stack in DFS (Part 1)

In Depth First Search, the system's call stack manages the state. Each time a node is visited, a new function frame is pushed onto the stack. If a graph is extremely deep, this can lead to a `RecursionError` or `StackOverflow`.

> **Optimization:** For industrial-scale graphs, an iterative DFS using a `collections.deque` as a manual stack is often safer than the recursive approach.

BFS and Shortest Paths

One of the most powerful properties of BFS is that in an unweighted graph, the first time you reach a node, you have found the absolute shortest path to it. This is because BFS explores all nodes at distance 1, then all at distance 2, and so on.

Deep Dive: The Call Stack in DFS (Part 2)

In Depth First Search, the system's call stack manages the state. Each time a node is visited, a new function frame is pushed onto the stack. If a graph is extremely deep, this can lead to a `RecursionError` or `StackOverflow`.

> **Optimization:** For industrial-scale graphs, an iterative DFS using a `collections.deque` as a manual stack is often safer than the recursive approach.

BFS and Shortest Paths

One of the most powerful properties of BFS is that in an unweighted graph, the first time you reach a node, you have found the absolute shortest path to it. This is because BFS explores all nodes at distance 1, then all at distance 2, and so on.